

Package ‘mathr’

August 31, 2026

Version 0.1.0

Title Scientific Computation using R

Description This is a collection of undergraduate level mathematical concepts essential for working at quantitative area. This includes calculus, distribution functions, random variate generation, linear algebra, differential equation, and optimization, but is suitable for one semester. Successor to the 'math' package, with the special and distribution functions reimplemented from freely licensed sources.

Depends R (>= 3.0.0)

Author Kyun-Seop Bae [aut, cre]

Maintainer Kyun-Seop Bae <k@acr.kr>

Copyright 2018-, Kyun-Seop Bae

License GPL-3

NeedsCompilation no

LazyLoad yes

URL <https://github.com/ksbae/mathr>

BugReports <https://github.com/ksbae/mathr/issues>

Contents

betacf	3
betai	4
betaiapprox	5
Bin2Dec	6
ChkFx	7
Chol	8
Choose	9
conv	9
conv0	10
Dbeta	11
Dbinom	11
dc	12
Dchisq	13
Dec2Bin	13
deconv	14
deconv0	15

DENORM	15
Deriv0	16
Deriv1	16
Deriv2	17
Df	18
Dgamma	18
Dlnorm	19
Dnorm	20
Dpois	20
Dt	21
ellipRange	22
ellipse	22
erf	23
erfc	24
erfccheb	25
evJacobi	26
EXP	26
GAMMA	27
gammln	28
gammq	28
gammpprox	29
gammq	30
gcf	30
GenD	31
GetP	32
GQuad8	32
Grad	33
gser	34
Hessian	34
invbetai	35
inverfc	36
invgammp	36
LDLT	37
LGAMMA	38
LOG	38
MachEps	39
MachEps2	40
MachEps3	40
MachEps4	41
mlr	42
NashSVD	43
NORM	43
Optim0	44
OuterProd	45
Pbeta	45
Pbinom	46
Pchisq	46
Pf	47
Pgamma	48
Plnorm	48
Pnorm	49
PolyNom3	49

Ppois	50
Pt	51
Qbeta	51
Qbinom	52
Qchisq	52
Qf	53
Qgamma	54
Qlnorm	54
Qnorm	55
Qpois	55
Qt	56
Rbeta	57
Rexp	57
Rgamma	58
Rgamma0	59
RK4	59
Rmvn	60
Rnorm	61
romb	61
Round	62
run.p	63
Run.test	63
runifLC	64
sft	65
simps13	65
simps38	66
SQRT	67
SymDet	67
SymInv	68
SymSol	68
tableFactorial	69
trapez0	69
trapez1	70
VMmin	71
Index	72

betacf

*Continued fraction approximation of incomplete beta function***Description**

Incomplete beta function with parameter a and b

Usage

betacf(a, b, x)

Arguments

a	shape constant, distribution parameter
b	shape constant, distribution parameter
x	to which it integrates

Details

Ref: Abramowitz and Stegun 26.5.8; beta_cont_frac of the GNU Scientific Library, specfunc/beta_inc.c

Value

integrated value

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
 M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

 betai

Incomplete beta function

Description

Incomplete beta function with parameter a and b

Usage

betai(a, b, x)

Arguments

a	shape constant, distribution parameter
b	shape constant, distribution parameter
x	to which it integrates

Details

This calls betaiapprox, betacf and/or gammln. Ref: gsl_sf_beta_inc of the GNU Scientific Library, specfunc/beta_inc.c

Value

integrated value

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

betaiapprox

Quadrature approximation of incomplete beta function

Description

Retained for backward compatibility. Forwards to betai

Usage

betaiapprox(a, b, x)

Arguments

a	shape constant, distribution parameter
b	shape constant, distribution parameter
x	to which it integrates

Details

Retained for backward compatibility. The continued fraction of betacf converges over the whole range once the symmetry transform is applied, so the separate large-parameter quadrature branch is no longer needed.

Value

integrated value

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

Bin2Dec

*Binary to Decimal Format***Description**

Transforms a binary IEEE 754 format value stored in an array to a decimal value.

Usage

```
Bin2Dec(b)
```

Arguments

b a binary value stored in an array with IEEE 754 format

Details

IEEE 754 defined how the computer present float point number in a binary format. This transforms binary format to decimal value.

Value

A decimal value with attribute "Expression"

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

IEEE 754

Examples

```
Bin2Dec(rep(0, 32))           # +0.0
Bin2Dec(c(1, rep(0, 31)))     # -0.0
Bin2Dec(c(0, rep(1, 8), rep(0, 23))) # +Inf
Bin2Dec(c(1, rep(1, 8), rep(0, 23))) # -Inf
Bin2Dec(c(0, rep(1, 8), rep(1, 23))) # NaN
Bin2Dec(c(1, rep(1, 8), rep(1, 23))) # NaN
Bin2Dec(c(1, rep(1, 30), 0))   # NaN

Bin2Dec(c(0, 0,1,1,1,1,1,1,1,1,1, 0,0,0,0, rep(0,48))) # +1
Bin2Dec(c(1, 0,1,1,1,1,1,1,1,1,1, 0,0,0,0, rep(0,48))) # -1
Bin2Dec(c(0, 0,1,1,1,1,1,1,1,1,1, 1,0,0,0, rep(0,48))) # 1.5
Bin2Dec(c(0, 1,0,0,0,0,0,0,0,0,0,0, 0,0,0,0, rep(0,48))) # 2
Bin2Dec(c(0, 1,0,0,0,0,0,0,0,0,0,1, 1,0,1,0, rep(0,48))) # 6.5
```

ChkFx

*Check function input and output types***Description**

Check the input and output of a function be a scalar, a vector, or a matrix.

Usage

```
ChkFx(func, x)
```

Arguments

func	function to be checked
x	given input

Details

This is to check the types (scalar, vector, matrix) of a function input and output.

Value

CaseNo	Case number
minInput	minimal input type
minOutput	minimal output type
Given	type of given value
Repeat	repeated evaluation of the function
Gradient	type of gradient output

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx0 = function(x) {
  if (length(x) > 1) return(NULL)
  return(x*x)
}

fx1 = function(x) x*x # independent each other (elementwise operation only)
fx2 = function(x) sum(x)
fx3 = function(x) c(sum(x), mean(x)) # c(mean(x), sd(x))

fx4 = function(x) {
  if (is.vector(x)) { Result = sum(x)
  } else if (is.matrix(x)) { Result = rowSums(x)
  } else { Result = NULL }
  return(Result)
}
```

```
fx5 = function(x) {  
  if (is.vector(x)) { Result = c(sum(x), mean(x))  
  } else if (is.matrix(x)) { Result = cbind(rowSums(x), rowMeans(x))  
  } else { Result = NULL }  
  return(Result)  
}  
  
ChkFx(fx1, 3)  
ChkFx(fx1, 1:3)  
ChkFx(fx2, 3)  
ChkFx(fx2, 1:3)  
ChkFx(fx3, 3)  
ChkFx(fx3, 1:3)  
ChkFx(fx4, 1:3)  
ChkFx(fx4, matrix(1:9, nrow=3))  
ChkFx(fx5, 1:3)  
ChkFx(fx5, matrix(1:9, nrow=3))
```

Chol*Cholesky Decomposition*

Description

Cholesky decomposition of a symmetric matrix

Usage

Chol(M)

Arguments

M a symmetric matrix of real values

Details

Input should be a symmetric matrix of real values. This is one-to-one transformation.

Value

a decomposed matrix

Author(s)

Kyun-Seop Bae <k@acr.kr>

Choose	<i>Combination function</i>
--------	-----------------------------

Description

Another simple implementation of combination function like choose

Usage

Choose(n, r)

Arguments

n	total count
r	cont to choose

Details

This is simple but not easily overflowed version of combination function.

Value

combination of choosing r cases from n cases.

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
Choose(10, 5)
Choose(100, 50)
Choose(1000, 500)
Choose(2000, 1000)
```

conv	<i>Convolution using fft</i>
------	------------------------------

Description

Convolute two given sequences and returns real part only.

Usage

conv(x, y)

Arguments

x	Input sequence
y	Response sequence to unit impulse

Details

One is input and the other is unit response sequence. The lengths of two sequences should be same. The order is not important.

Value

Real part of convolution result.

Author(s)

Kyun-Seop Bae <k@acr.kr>

conv0	<i>Convolution using fft</i>
-------	------------------------------

Description

Convolute two given sequences.

Usage

```
conv0(x, y)
```

Arguments

x	Input sequence
y	Response sequence to unit impulse

Details

One is input and the other is unit response sequence. The lengths of two sequences should be same. The order is not important

Value

convolution result.

Author(s)

Kyun-Seop Bae <k@acr.kr>

Dbeta	<i>Density of beta distribution</i>
-------	-------------------------------------

Description

Calculate the density of a beta distribution

Usage

```
Dbeta(x, alph, bet)
```

Arguments

x	x value for a beta distribution
alph	the shape parameter alpha of a beta distribution
bet	the shape parameter beta of a beta distribution

Details

It calculates the density for x value of a beta distribution.

Value

density for x

Author(s)

Kyun-Seop Bae <k@acr.kr>

Dbinom	<i>Density of binomial distribution</i>
--------	---

Description

Calculate the mass for n of a binomial distribution

Usage

```
Dbinom(k, n, pe)
```

Arguments

k	count k of wanted event for a binomial distribution
n	total number of trials n of a binomial distribution
pe	the probability of wanted event for a binomial distribution

Details

It calculates the probability for count k of a binomial distribution.

Value

probability for k

Author(s)

Kyun-Seop Bae <k@acr.kr>

dc	<i>deconvolution</i>
----	----------------------

Description

deconvollute two given sequences measured at different time points.

Usage

dc(t1, c1, t2, c2)

Arguments

t1	time points of sequence 1
c1	measurement values of sequence 1, to be deconvoluted, Response values
t2	time points of sequence 2
c2	measurement values of sequence 2. input sequence or unit reponse sequence

Details

Two measurement sequences with two different time point sequences. The order matters.

Value

Deconvolution result: input sequence or unit response sequence.

Author(s)

Kyun-Seop Bae <k@acr.kr>

Dchisq*Density of chi-square distribution*

Description

Calculate the density of a chi-square distribution

Usage

```
Dchisq(x2, nu)
```

Arguments

x2	chi-square value for a chi-square distribution
nu	the shape parameter nu of a chi-square distribution

Details

It calculates the density for chi-sqaure value of a chi-square distribution.

Value

density for x2

Author(s)

Kyun-Seop Bae <k@acr.kr>

Dec2Bin*Decimal to Binary Format Vector*

Description

Transforms a decimal value to a binary IEEE 754 format value stored in an array.

Usage

```
Dec2Bin(x, Double=TRUE)
```

Arguments

x	a decimal value
Double	set this FALSE if you want single precision.

Details

IEEE 754 defined how the computer present float point number in a binary format. This transforms a decimal value to a binary format array.

Value

An array containing the binary value.

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

IEEE 754

Examples

```
Dec2Bin(1)
Dec2Bin(-1)
Dec2Bin(1.5)
Dec2Bin(2)
Dec2Bin(6.5)
```

```
Dec2Bin(1, FALSE)
Dec2Bin(-1, FALSE)
Dec2Bin(1.5, FALSE)
Dec2Bin(2, FALSE)
Dec2Bin(6.5, FALSE)
```

deconv

Deconvolution using fft

Description

Deconvolute two given sequences, and return real part only.

Usage

```
deconv(z, x)
```

Arguments

z	Response sequence to be deconvoluted
x	Input or Unit response sequence

Details

The lengths of two sequences should be same. The order matters.

Value

Real part of deconvolution result.

Author(s)

Kyun-Seop Bae <k@acr.kr>

deconv0	<i>Deconvolution using fft</i>
---------	--------------------------------

Description

Deconvolute two given sequences.

Usage

deconv0(z, x)

Arguments

z	Response sequence to be deconvoluted
x	Input or Unit response sequence

Details

The lengths of two sequences should be same. The order matters.

Value

Deconvolution result.

Author(s)

Kyun-Seop Bae <k@acr.kr>

DENORM	<i>DENORM</i>
--------	---------------

Description

From normalized number to decimal number

Usage

DENORM(Norm)

Arguments

Norm	Normalized number, a vector of length 2.
------	--

Details

Ref. Plauger PJ The Standard C Library. 1992.

Value

Norm[1] * 2^Norm[2]

Author(s)

Kyun-Seop Bae <k@acr.kr>

Deriv0

First derivative, version 0

Description

Get first derivative of scalar-input scalar-valued function using Richardson extrapolation 4th order.

Usage

Deriv0(fx, x)

Arguments

fx	scalar valued function
x	x point at which derivative is calculated

Details

This simplest implementation of 4th order Richardson extrapolation for derivative calculation uses a 4x4 matrix.

Value

derivative value

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx = function(x) x*x
Deriv0(fx, 1) # 2
```

Deriv1

First derivative, version 1

Description

Get first derivative of scalar-input scalar-valued function using Richardson extrapolation 4th order.

Usage

Deriv1(fx, x)

Arguments

fx	scalar valued function
x	x point at which derivative is calculated

Details

This implementation of 4th order Richardson extrapolation for derivative calculation, uses a vector of length 4.

Value

derivative value

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx = function(x) x*x
Deriv1(fx, 1) # 2
```

Deriv2

First derivative, version 2

Description

Get first derivative of vector-input (multi-variable) scalar-valued function using Richardson extrapolation 4th order.

Usage

```
Deriv2(fx, x)
```

Arguments

fx	vector-input (multi-variable) scalar-valued function
x	vector x at which gradients are calculated

Details

This implementation of 4th order Richardson extrapolation for derivative calculation, uses 4 vectors.

Value

gradient values

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx2 = function(x) sum(x*x)
Deriv2(fx2, c(1,1)) # 2, 2
```

Df

*Density of F distribution***Description**

Calculate the density of a F distribution

Usage

```
Df(f, nu1, nu2)
```

Arguments

f	f value for an F distribution
nu1	the shape parameter nu1 of an F distribution
nu2	the shape parameter nu2 of an F distribution

Details

It calculates the density for f value of an F distribution.

Value

density for f

Author(s)

Kyun-Seop Bae <k@acr.kr>

Dgamma

*Density of gamma distribution***Description**

Calculate the density of a gamma distribution

Usage

```
Dgamma(x, alph, bet=1)
```

Arguments

x	x value for a gamma distribution
alph	the shape parameter alpha of a gamma distribution
bet	the shape parameter beta of a gamma distribution

Details

It calculates the density for x value of a gamma distribution.

Value

density for x

Author(s)

Kyun-Seop Bae <k@acr.kr>

Dlnorm	<i>Density of log-normal distribution</i>
--------	---

Description

Calculate the density of a log-normal distribution

Usage

```
Dlnorm(x, mu=0, sig=1)
```

Arguments

x	x value for a log-normal distribution
mu	the shape parameter mu of a log-normal distribution
sig	the shape parameter sigma of a log-normal distribution

Details

It calculates the density for x value of a log-normal distribution.

Value

density for x

Author(s)

Kyun-Seop Bae <k@acr.kr>

Dnorm	<i>Density of normal distribution</i>
-------	---------------------------------------

Description

Calculate the density of a normal distribution

Usage

```
Dnorm(x, mu=0, sig=1)
```

Arguments

x	x value for a normal distribution
mu	the shape parameter mu of a normal distribution
sig	the shape parameter sigma of a normal distribution

Details

It calculates the density for x value of a normal distribution.

Value

density for x

Author(s)

Kyun-Seop Bae <k@acr.kr>

Dpois	<i>Density of Poisson distribution</i>
-------	--

Description

Calculate the mass for n of a Poisson distribution

Usage

```
Dpois(n, lam)
```

Arguments

n	count n for a Poisson distribution
lam	shape parameter lambda of a Poisson distribution

Details

It calculates the mass for n of a Poisson distribution.

Value

mass (probability) for n

Author(s)

Kyun-Seop Bae <k@acr.kr>

Dt	<i>Density of t distribution</i>
----	----------------------------------

Description

Calculate the density of a t distribution

Usage

Dt(t, nu, mu=0, sig=1)

Arguments

t	t value for a t distribution
nu	degree of freedom
mu	the shape parameter mu of a t distribution
sig	the shape parameter sigma of a t distribution

Details

It calculates the density for t value of a t distribution.

Value

density for t

Author(s)

Kyun-Seop Bae <k@acr.kr>

 ellipRange

EllipRange

Description

Ranges of an ellipse

Usage

```
ellipRange(center=c(0, 0), radius=c(2, 1), alpha=0)
```

Arguments

center	coordinate of center point
radius	length of long and short axes
alpha	rotation angle of long axis

Details

This returns ranges of x and y axes.

Value

Column vectors of x and y ranges

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
ellipRange()
ellipRange(c(0, 0), c(3, 2), pi/4)
ellipRange(c(1, 1), c(3, 2), pi/4)
ellipRange(c(1, 1), c(3, 2), pi/6)
```

 ellipse

Ellipse

Description

Plot ellipse and return coordinates.

Usage

```
ellipse(center=c(0, 0), radius=c(2, 1), alpha=0, npoints=100, add=F, ...)
```

Arguments

center	coordinate of center point
radius	length of long and short axes
alpha	rotation angle of long axis
npoints	number of points used to plot
add	whether to plot on an active device
...	arguments to be passed to lines or plot

Details

This plots an ellipse and returns coordinates of points used to plot.

Value

coordinates of points used to plot.

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
dev.new()
ellipse(asp=1)

dev.new()
ellipse(c(0, 0), c(3, 2), pi/4, asp=1)

dev.new()
ellipse(c(1, 1), c(3, 2), pi/4, asp=1)
points(1, 1, pch="+")
abline(v=1, h=1, lty=3)
```

erf

Error function

Description

Error function.

Usage

```
erf(x)
```

Arguments

x	a real value
---	--------------

Details

This calls `erfccheb`.

Ref: $\text{erf}(x) = P(1/2, x^2)$, the relation the GNU Scientific Library uses between `gsl_sf_erf` and `gsl_sf_gamma_inc_P`

Value

Integration value $\frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
 M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

erfc	<i>Complementary error function</i>
------	-------------------------------------

Description

Complementary error function.

Usage

`erfc(x)`

Arguments

x a real value

Details

$1 - \text{erf}(x)$

This calls `erfccheb`.

Ref: $\text{erfc}(x) = Q(1/2, x^2)$, the relation the GNU Scientific Library uses between `gsl_sf_erfc` and `gsl_sf_gamma_inc_Q`

Value

$1 - \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

erfccheb

Chebyshev approximation for error function

Description

Retained for backward compatibility. Forwards to erfc.

Usage

erfccheb(z)

Arguments

z a real value

Details

Retained for backward compatibility. Forwards to erfc. This is called by erf and erfc . Retained
for backward compatibility. Forwards to erfc, which now uses the incomplete gamma relation over
the whole range.

Value

Approximation value for error function

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

evJacobi

Spectral Decomposition of a Real Symmetric Matrix by Jacobi Algorithm

Description

Eigen values and vectors of a Real Symmetric Matrix by Jacobi Algorithm

Usage

evJacobi(A)

Arguments

A a symmetric matrix of real values

Details

Input should be a symmetric matrix of real values.

Value

values eigen values
vectors eigen vectors

Author(s)

Kyun-Seop Bae <k@acr.kr>

EXP

Exponential

Description

Exponential of a real value.

Usage

EXP(x)

Arguments

x a real number to be Exponentiated

Details

Ref. Plauger PJ The Standard C Library. 1992.

Value

Exponential value of a real value.

Author(s)

Kyun-Seop Bae <k@acr.kr>

GAMMA

Gamma Function

Description

This gamma function is a generalization of factorial working with real numbers. This function uses Lanczos' approximation and fairly short.

Usage

GAMMA(z)

Arguments

z real number

Details

If x is an integer, $\text{GAMMA}(x) = (x - 1)!$. If x is a negative integer, it returns +Inf.

Value

Gamma function real value

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
GAMMA(0)
GAMMA(1) # 0! = 1
GAMMA(2) # 1! = 1
GAMMA(171) # 170! = 7.257416e+306
format(GAMMA(0.5), digits=22) # = sqrt(pi) = 1.772453850905516
format(gamma(0.5), digits=22)
format(sqrt(pi), digits=22)
format(GAMMA(1.5), digits=22)
format(gamma(1.5), digits=22)
```

gammln	<i>Log gamma function</i>
--------	---------------------------

Description

Log gamma function

Usage

gammln(xx)

Arguments

xx a real positive number to be gammln-ed

Details

Ref: Lanczos g=7 coefficients as used by the GNU Scientific Library, gsl_sf_lngamma

Value

Log gamma

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
 M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

gammp	<i>Incomplete gamma function</i>
-------	----------------------------------

Description

Incomplete gamma function with parameter a

Usage

gammp(a, x)

Arguments

a shape constant, distribution parameter
 x to which it integrates

Details

This calls gser or gampapprox. Ref: Cephes igam/igamc, as redistributed in ALGLIB special-functions.cs

Value

integrated value

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

gampapprox

Quadrature approximation of incomplete gamma function

Description

Retained for backward compatibility. Forwards to gamp or gammq

Usage

```
gampapprox(a, x, psig = 1)
```

Arguments

a	shape constant, distribution parameter
x	to which it integrates
psig	1 for P(a, x), 0 for Q(a, x) = 1 - P(a, x)

Details

This calls gamma1n. Retained for backward compatibility. The Cephes series and continued fraction converge over the whole range, so the separate large-a quadrature branch is no longer needed.

Value

integrated value

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

gammq	<i>Incomplete gamma function</i>
-------	----------------------------------

Description

Incomplete gamma function, $Q(a, x) = 1 - P(a, x)$

Usage

`gammq(a, x)`

Arguments

a	shape constant, distribution parameter
x	to which it integrates

Details

This calls `gser` or `gampapprox`. Ref: Cephes `igam/igamc`, as redistributed in ALGLIB special-functions.cs

Value

integrated value

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
 M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

gcf	<i>Continued fraction approximation of incomplete gamma function</i>
-----	--

Description

Continued fraction approximation of incomplete gamma function, $Q(a, x)$

Usage

`gcf(a, x)`

Arguments

a	shape constant, distribution parameter
x	to which it integrates

Details

This calls `gamma1n`. Ref: Abramowitz and Stegun 6.5.31; Cephes `igamc`, as redistributed in ALGLIB `specialfunctions.cs`

Value

integrated value

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

 GenD

Get Hessian and Gradient

Description

This is the combination of Grad and Hessian

Usage

`GenD(func, x)`

Arguments

<code>func</code>	vector-input (multi-variable) scalar-valued function
<code>x</code>	x point at which gradients are calculated

Details

Input is the same to Grad and Hessian functions.

Value

<code>gr</code>	gradient vector
<code>Hessian</code>	Hessian matrix

Author(s)

Kyun-Seop Bae <k@acr.kr>

GetP

Get the Direction Vector

Description

Get the direction vector for the new line search during the minimization.

Usage

GetP(Hv, g)

Arguments

Hv	vector of lower triangular part of hessian
g	gradient

Details

The result is same with solve(H, -g)

Value

Direction vector for a new search

Author(s)

Kyun-Seop Bae <k@acr.kr>

GQuad8

Integration using Gaussian quadrature

Description

Integrates with given fx, a, b, and n

Usage

GQuad8(fx, a, b)

Arguments

fx	real valued scalar function
a	from
b	to

Details

This gives approximate integration value using Gaussian quadrature. It uses 8 points approximation.

Value

Integration value from a to b with function.

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx = function(x) exp(-x/100)
GQuad8(fx, 0, 24)
```

Grad	<i>Gradient Calculation</i>
------	-----------------------------

Description

Get gradient using Richardson extrapolation 4th order.

Usage

```
Grad(func, x)
```

Arguments

func	vector-input (multi-variable) scalar-valued function
x	x point at which gradients are calculated

Details

This is a simplified implementation of 4th order Richardson extrapolation for gradient calculation in numDeriv package.

Value

gradient value

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx2 = function(x) sum(x*x)
Grad(fx2, c(1, 1)) # 2, 2
```

gser	<i>Series approximation of incomplete gamma function</i>
------	--

Description

Series approximation of incomplete gamma function, $P(a, x)$

Usage

`gser(a, x)`

Arguments

a	shape constant, distribution parameter
x	to which it integrates

Details

This calls `gamma1n`. Ref: Abramowitz and Stegun 6.5.29; Cephes igam, as redistributed in ALGLIB `specialfunctions.cs`

Value

integrated value

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
 M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

Hessian	<i>Hessian Calculation</i>
---------	----------------------------

Description

Get hessian using Richardson extrapolation 4th order.

Usage

`Hessian(fx, x)`

Arguments

fx	vector-input (multi-variable) scalar-valued function
x	x point at which hessian is calculated

Details

This is a simplified implementation of 4th order Richardson extrapolation for hessian calculation. See numDeriv package for more detail.

Value

hessian value

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx2 = function(x) sum(x*x)
Hessian(fx2, c(1, 1)) # matrix(c(2, 0, 0, 2), nrow=2)
```

invbetai	<i>Inverse of gammap function</i>
----------	-----------------------------------

Description

Inverse of incomplete beta function with parameter a and b

Usage

```
invbetai(p, a, b)
```

Arguments

p	integration value between 0 and 1
a	shape constraint, distribution parameter
b	shape constraint, distribution parameter

Details

This calls gammln and betai Ref: bracketed Newton steps on betai, following gsl_cdf_beta_Pinv of the GNU Scientific Library

Value

inverse of incomplete beta function

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
 M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

inverfc	<i>Inverse of complementary error function</i>
---------	--

Description

Inverse of complementary error function.

Usage

inverfc(p)

Arguments

p	a positive real integration value usually between 0 and 2
---	---

Details

This calls erfc.

Ref: Cephes ndtri, as redistributed in ALGLIB specialfunctions.cs (invnormalcdf)

Value

inverse of complementary error function value, $p = \text{erfc}(x)$

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz
 M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

invgammp	<i>Inverse of gammp function</i>
----------	----------------------------------

Description

Inverse of gammp function

Usage

invgammp(p, a)

Arguments

p	integration value between 0 and 1
a	shape constrant, distribution parameter

Details

This calls `gammln` and `gamp` Ref: Wilson and Hilferty (1931) starting value refined by bracketed Newton steps, following `gsl_cdf_gamma_Pinv` of the GNU Scientific Library

Value

inverse of incomplete gamma function

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Galassi M, et al. GNU Scientific Library Reference Manual. 3rd ed. Network Theory, 2009.
 Moshier SL. Methods and Programs for Mathematical Functions. Ellis Horwood, 1989. Abramowitz M, Stegun IA. Handbook of Mathematical Functions. Dover, 1972.

LDLT	<i>LDL' Transformation</i>
------	----------------------------

Description

LDL' factorization of a symmetric matrix

Usage

LDLT(SymMat)

Arguments

SymMat a symmetric matrix of real values

Details

Input should be a symmetric matrix of real values.

Value

L lower triangular matrix
 D diagonal matrix

Author(s)

Kyun-Seop Bae <k@acr.kr>

 LGAMMA

Natural Log Gamma Function

Description

This log gamma function is for a larger value of real numbers. This function uses Lanczos' approximation and fairly short.

Usage

LGAMMA(z)

Arguments

z real number

Details

If abs(z) is less than or equal to 171, it uses GAMMA function.

Value

natural log value of Gamma function

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
LGAMMA(0)
LGAMMA(1) # log(0!) = 0
LGAMMA(2) # log(1!) = 0
LGAMMA(171) # log(170!) = 706.5731
LGAMMA(-171) # Inf for all negative integers, as in base lgamma()
format(LGAMMA(0.5), digits=22) # = log(sqrt(pi)) = 0.5723649429247
format(lgamma(0.5), digits=22)
format(log(sqrt(pi)), digits=22)
format(LGAMMA(1.5), digits=22)
format(lgamma(1.5), digits=22)
```

 LOG

Logarithm natural or common

Description

Natural or common log of a real positive value.

Usage

LOG(x, DecFlag=FALSE)

Arguments

x a real positive number to be log-ed
DecFlag FALSE for natural log, TRUE for common log

Details

Ref. Plauger PJ The Standard C Library. 1992.

Value

log value of base *e* or 10.

Author(s)

Kyun-Seop Bae <k@acr.kr>

MachEps

Get Machine Epsilon

Description

Calculate machine epsilon.

Usage

MachEps()

Arguments

none

Details

Machine epsilon + 1 is different from 1. But half of it added to one cannot be differentiated from one.

Value

Machine epsilon

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

MachEps()

MachEps2*Get Epsilon a little more general than Machine Epsilon*

Description

Calculate epsilon using x and Start value.

Usage

```
MachEps2(x = 1, Start = 1)
```

Arguments

x	value to be used at the equation of $x + \text{eps} > x$
Start	Starting epsilon value. Epsilon is halved by each iteration.

Details

This finds the smallest epsilon which satisfies $x + \text{eps} > x$ by halving epsilon for each iteration.

Value

epsilon

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
MachEps2(100, 1)
(eps2 = MachEps2(1, 0.8))
1 + eps2 == 1 # FALSE and less than .Machine$double.eps
```

MachEps3*Get Meticulous Positive Epsilon*

Description

Calculate meticulous positive epsilon using bisectional search.

Usage

```
MachEps3(MaxIter = 1000)
```

Arguments

MaxIter	maximum iteration number
---------	--------------------------

Details

This finds really the smallest epsilon by bisectional search.

Value

epsilon with "iteration" count attribute

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
MachEps3() # Smallest epsilon in this environment. Smaller than .Machine$double.eps
```

MachEps4	<i>Get Meticulous Negative Epsilon</i>
----------	--

Description

Calculate meticulous negative epsilon using bisectional search.

Usage

```
MachEps4(MaxIter = 1000)
```

Arguments

MaxIter maximum iteration number

Details

This finds really the smallest epsilon by bisectional search.

Value

epsilon with "iteration" count attribute

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
MachEps4() # different not just sign, but also magnitude with MachEps3()
```

`mlr`*Multiple Linear Regression*

Description

Multiple linear regression.

Usage

```
mlr(y, x.raw, standardize=0, Plot=FALSE)
```

Arguments

<code>y</code>	dependent part, a vector
<code>x.raw</code>	independent part, a matrix
<code>standardize</code>	standardize method, 0 means no standardize
<code>Plot</code>	plot or not

Details

It shows various outputs.

Value

Variable	variable names
Estimate	estimates
SE	standard error
T	t value
p-value	p-value
Residual	residuals
R-student	externally studentized residuals
hat	hat value
Cook's D	Cook's D
Covratio	Covratio
DIFFITS	DIFFITS
DFBETAs	DFBETAs

Author(s)

Kyun-Seop Bae <k@acr.kr>

NashSVD	<i>Singular Value Decomposition by Nash</i>
---------	---

Description

Singular value decomposition by Nash

Usage

NashSVD(M)

Arguments

M a symmetric matrix of real values

Details

This can be used for a least square problem.

Value

Decomposed result

Author(s)

Kyun-Seop Bae <k@acr.kr>

NORM	<i>NORM</i>
------	-------------

Description

From a decimal number to a normalized number

Usage

NORM(x)

Arguments

x a decimal number to be normalized

Details

It returns (f,n) for $x = f * 2^n$ where $(0.5 \leq f \leq 1)$. Returns a vector of length 2. Ref. Plauger PJ The Standard C Library. 1992.

Value

Norm[1] * 2^Norm[2]

Author(s)

Kyun-Seop Bae <k@acr.kr>

Optim0

*Prototype of optimization function***Description**

To show the algorithm how quasi-Newton type optimization algorithm works.

Usage

```
Optim0(x0, func, TypF=1, GradTol=.Machine$double.eps^(1/3),
      StepTol=.Machine$double.eps^(2/3),
      FnTol=.Machine$double.eps^(1/3), ItnLimit=100)
```

Arguments

x0	initial value
func	function to be minimized
TypF	typical function value, a scalar, to measure approximate magnitude
GradTol	gradient tolerance
StepTol	x direction or step tolerance
FnTol	y or function tolerance
ItnLimit	iteration count limit

Details

If the changes of between steps are less than the tolerances, it stops.

Value

par	final x vector values
value	function value at the final x vector
FnCount	function evaluation count
convergence	convergence is met or not
grad	gradient at the final x vector
hessian	hessian at the final x vector
RelGrad	relative gradient
RelStep	relative step
DelF	delta F, function value
f	function values
x	x vector values

Author(s)

Kyun-Seop Bae <k@acr.kr>

OuterProd*Outer product of two three-dimensional vectors*

Description

Calculate the outer product of two three-dimensional vectors

Usage

```
OuterProd(a, b)
```

Arguments

a	the first vector
b	the second vector

Details

Outer product is not commutative.

Value

Outer product of two three-dimension vectors

Author(s)

Kyun-Seop Bae <k@acr.kr>

Pbeta*Cumulative probability of beta distribution*

Description

Calculate the cumulative probability for x of a beta distribution

Usage

```
Pbeta(x, alph, bet)
```

Arguments

x	x value for a beta distribution
alph	the shape parameter alpha of a beta distribution
bet	the shape parameter beta of a beta distribution

Details

It calculates the cumulative probability for x of a beta distribution.

Value

cumulative probability till x

Author(s)

Kyun-Seop Bae <k@acr.kr>

Pbinom

Cumulative probability of Poisson distribution

Description

Calculate the cumulative probability $P(X \leq k)$ of a binomial distribution

Usage

Pbinom(k, n, pe)

Arguments

k	count k of wanted event for a binomial distribution
n	total number of trials n of a binomial distribution
pe	the probability of wanted event for a binomial distribution

Details

It calculates the cumulative probability for k of a Poisson distribution.

Value

cumulative probability till k

Author(s)

Kyun-Seop Bae <k@acr.kr>

Pchisq

Cumulative probability of chi-square distribution

Description

Calculate the cumulative probability for x2 of a chi-square distribution

Usage

Pchisq(x2, nu)

Arguments

x2	chi-square value for a chi-square distribution
nu	the shape parameter nu of a chi-square distribution

Details

It calculates the cumulative probability for x2 of a chi-square distribution.

Value

cumulative probability till x2

Author(s)

Kyun-Seop Bae <k@acr.kr>

Pf

Cumulative probability of F distribution

Description

Calculate the cumulative probability for f of an F distribution

Usage

Pf(f, nu1, nu2)

Arguments

f	f value for an F distribution
nu1	the shape parameter nu1 of an F distribution
nu2	the shape parameter nu2 of an F distribution

Details

It calculates the cumulative probability for f of an F distribution.

Value

cumulative probability till f

Author(s)

Kyun-Seop Bae <k@acr.kr>

Pgamma	<i>Cumulative probability of gamma distribution</i>
--------	---

Description

Calculate the cumulative probability for x of a gamma distribution

Usage

```
Pgamma(x, alph, bet=1)
```

Arguments

x	x value for a gamma distribution
alph	the shape parameter alpha of a gamma distribution
bet	the shape parameter beta of a gamma distribution

Details

It calculates the cumulative probability for x of a gamma distribution.

Value

cumulative probability till x

Author(s)

Kyun-Seop Bae <k@acr.kr>

Plnorm	<i>Cumulative probability of log-normal distribution</i>
--------	--

Description

Calculate the cumulative probability for t of a log-normal distribution

Usage

```
Plnorm(x, mu=0, sig=1)
```

Arguments

x	x value for a log-normal distribution
mu	the shape parameter mu of a log-normal distribution
sig	the shape parameter sigma of a log-normal distribution

Details

It calculates the cumulative probability for x of a log-normal distribution.

Value

cumulative probability till x

Author(s)

Kyun-Seop Bae <k@acr.kr>

Pnorm	<i>Cumulative probability of normal distribution</i>
-------	--

Description

Calculate the cumulative probability for t of a normal distribution

Usage

```
Pnorm(x, mu=0, sig=1)
```

Arguments

x	x value for a normal distribution
mu	the shape parameter mu of a normal distribution
sig	the shape parameter sigma of a normal distribution

Details

It calculates the cumulative probability for x of a normal distribution.

Value

cumulative probability till x

Author(s)

Kyun-Seop Bae <k@acr.kr>

PolyNom3	<i>Approximation of pnorm using a polynomial equation</i>
----------	---

Description

It approximates pnorm using a polynomial equation.

Usage

```
PolyNom3(x)
```

Arguments

x	Z value
---	---------

Details

It calculates the approximation of `pnorm(x)`.

Value

Probability of $Z < x$

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
format(pnorm(2), digits=22)
format(PolyNom3(2), digits=22) # Compare with the above
```

Ppois	<i>Cumulative probability of Poisson distribution</i>
-------	---

Description

Calculate the cumulative probability $P(X \leq n)$ of a Poisson distribution

Usage

```
Ppois(n, lam)
```

Arguments

n	count n for a Poisson distribution
lam	shape parameter lambda of a Poisson distribution

Details

It calculates the cumulative probability for n of a Poisson distribution.

Value

cumulative probability till n

Author(s)

Kyun-Seop Bae <k@acr.kr>

Pt	<i>Cumulative probability of t distribution</i>
----	---

Description

Calculate the cumulative probability for t of a t distribution

Usage

```
Pt(t, nu, mu=0, sig=1)
```

Arguments

t	t value for a t distribution
nu	degree of freedom
mu	the shape parameter mu of a t distribution
sig	the shape parameter sigma of a t distribution

Details

It calculates the cumulative probability for t of a t distribution.

Value

cumulative probability till t

Author(s)

Kyun-Seop Bae <k@acr.kr>

Qbeta	<i>Reverse of Pbeta function</i>
-------	----------------------------------

Description

Calculate the x value for the cumulative probability p of a beta distribution

Usage

```
Qbeta(p, alph, bet)
```

Arguments

p	cumulative probability p for a beta distribution
alph	the shape parameter alpha of a beta distribution
bet	the shape parameter beta of a beta distribution

Details

It calculates the x value for the cumulative probability p of a beta distribution.

Value

x of $Pt(x, \alpha, \beta)$

Author(s)

Kyun-Seop Bae <k@acr.kr>

Qbinom

Reverse of Pbinom function

Description

Calculate the k for the cumulative probability p of a binomial distribution

Usage

Qbinom(p, n, pe)

Arguments

p	cumulative probability of wanted event for a binomial distribution
n	total number of trials n of a binomial distribution
pe	the probability of wanted event for a binomial distribution

Details

It calculates the k for the cumulative probability p of a Poisson distribution.

Value

k of $Pbinom(k, n, pe)$

Author(s)

Kyun-Seop Bae <k@acr.kr>

Qchisq

Reverse of Pchisq function

Description

Calculate the chi-square value for the cumulative probability p of a chi-square distribution

Usage

Qchisq(p, nu)

Arguments

p	cumulative probability p for a chi-square distribution
nu	the shape parameter nu of a chi-square distribution

Details

It calculates the chi-square value for the cumulative probability p of a chi-square distribution.

Value

x2 of $Pt(x2, nu)$

Author(s)

Kyun-Seop Bae <k@acr.kr>

Qf

Reverse of Pf function

Description

Calculate the f for the cumulative probability p of an F distribution

Usage

Qf(p, nu1, nu2)

Arguments

p	cumulative probability for f of an F distribution
nu1	the shape parameter nu1 of an F distribution
nu2	the shape parameter nu2 of an F distribution

Details

It calculates the f for the cumulative probability p of an F distribution.

Value

f of $Pf(f, nu1, nu2)$

Author(s)

Kyun-Seop Bae <k@acr.kr>

Qgamma	<i>Reverse of Pgamma function</i>
--------	-----------------------------------

Description

Calculate the x value for the cumulative probability p of a gamma distribution

Usage

```
Qgamma(p, alph, bet=1)
```

Arguments

p	cumulative probability p for a gamma distribution
alph	the shape parameter alpha of a gamma distribution
bet	the shape parameter beta of a gamma distribution

Details

It calculates the x value for the cumulative probability p of a gamma distribution.

Value

x of Pgamma(x, alph, bet)

Author(s)

Kyun-Seop Bae <k@acr.kr>

Qlnorm	<i>Reverse of Plnorm function</i>
--------	-----------------------------------

Description

Calculate the t value for the cumulative probability p of a log-normal distribution

Usage

```
Qlnorm(p, mu=0, sig=1)
```

Arguments

p	cumulative probability p for a log-normal distribution
mu	the shape parameter mu of a log-normal distribution
sig	the shape parameter sigma of a log-normal distribution

Details

It calculates the x value for the cumulative probability p of a log-normal distribution.

Value

x of Plnorm(x, mu, sigma)

Author(s)

Kyun-Seop Bae <k@acr.kr>

Qnorm

Reverse of Plnorm function

Description

Calculate the t value for the cumulative probability p of a normal distribution

Usage

Qnorm(p, mu=0, sig=1)

Arguments

p	cumulative probability p for a normal distribution
mu	the shape parameter mu of a normal distribution
sig	the shape parameter sigma of a normal distribution

Details

It calculates the x value for the cumulative probability p of a normal distribution.

Value

x of Pnorm(x, mu, sigma)

Author(s)

Kyun-Seop Bae <k@acr.kr>

Qpois

Reverse of Ppois function

Description

Calculate the n for the cumulative probability p of a Poisson distribution

Usage

Qpois(p, lam)

Arguments

p	cumulative probability of a Poisson distribution
lam	shape parameter lambda of a Poisson distribution

Details

It calculates the n for the cumulative probability p of a Poisson distribution.

Value

n of Ppois(n, lam)

Author(s)

Kyun-Seop Bae <k@acr.kr>

Qt	<i>Reverse of Pt function</i>
----	-------------------------------

Description

Calculate the t value for the cumulative probability p of a t distribution

Usage

Qt(p, nu, mu=0, sig=1)

Arguments

p	cumulative probability p for a t distribution
nu	degree of freedom
mu	the shape parameter mu of a t distribution
sig	the shape parameter sigma of a t distribution

Details

It calculates the t value for the cumulative probability p of a t distribution.

Value

t of Pt(t, nu, mu, sigma)

Author(s)

Kyun-Seop Bae <k@acr.kr>

Rbeta	<i>Random deviate from beta distribution</i>
-------	--

Description

Generate random numbers from a beta distribution

Usage

```
Rbeta(n, alph, bet)
```

Arguments

n	count, how many do you want
alph	shape parameter alpha of a beta distribution
bet	shape parameter beta of a beta distribution

Details

It generates random numbers from random numbers of other ditribution.

Value

a vector of random numbers

Author(s)

Kyun-Seop Bae <k@acr.kr>

Rexp	<i>Random deviate from exponential distribution</i>
------	---

Description

Generate random numbers from a exponential distribution

Usage

```
Rexp(n, alpha)
```

Arguments

n	count, how many do you want
alpha	shape parameter alpha of a exponential distribution

Details

It uses inverse-transformation method.

Value

a vector of random numbers

Author(s)

Kyun-Seop Bae <k@acr.kr>

Rgamma

Random deviate from gamma distribution

Description

Generate random numbers from a gamma distribution

Usage

```
Rgamma(n, alph, bet)
```

Arguments

n	count, how many do you want
alph	shape parameter alpha of a gamma distribution
bet	shape parameter beta of a gamma distribution

Details

It uses acceptance-rejection method.

Value

a vector of random numbers

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Banks J. Handbook of simulation (1998) pp152-153

Rgamma0

Random deviate from gamma distribution, simple method

Description

Generate random numbers from a gamma distribution by a simple method

Usage

```
Rgamma0(n, alph, bet)
```

Arguments

n	count, how many do you want
alph	shape parameter alpha of a gamma distribution
bet	shape parameter beta of a gamma distribution

Details

It uses acceptance-rejection method.

Value

a vector of random numbers

Author(s)

Kyun-Seop Bae <k@acr.kr>

References

Banks J. Handbook of simulation (1998) pp152-153

RK4

Runge-Kutta 4th Order Routine for Numeerical Integration

Description

Advance State vector by tau time with Deriv

Usage

```
RK4(State, tau, Deriv)
```

Arguments

State	state vector
tau	time amount to advance
Deriv	Derivative functions with the input of State vector

Details

Derive should accept State vector as an input.

Value

A new state vector after time advancement by tau

Author(s)

Kyun-Seop Bae <k@acr.kr>

Rmvn	<i>Random deviate from multivariate normal distribution</i>
------	---

Description

Generate random numbers from a multivariate normal distribution

Usage

```
Rmvn(n, Mu, Cov)
```

Arguments

n	count, how many do you want
Mu	mean vector of multivariate normal distribution
Cov	variance-covariance matrix of multivariate normal distribution

Details

It uses cholesky decomposition.

Value

a matrix of random vectors

Author(s)

Kyun-Seop Bae <k@acr.kr>

Rnorm	<i>Random deviate from normal distribution</i>
-------	--

Description

Generate random numbers from a normal distribution

Usage

```
Rnorm(n, mu=0, sigma=1)
```

Arguments

n	count, how many do you want
mu	mean of normal distribution
sigma	standard deviation of normal distribution

Details

It is more efficient with even n.

Value

a vector of random numbers

Author(s)

Kyun-Seop Bae <k@acr.kr>

romb	<i>Integration using Romberg integration</i>
------	--

Description

Integrates with given fx, a, b, and n

Usage

```
romb(fx, a, b, N=4)
```

Arguments

fx	real valued scalar function
a	from
b	to
N	degree; the larger, the precise

Details

This gives approximate integration value using Romberg integration. Argument N should larger than 2.

Value

Integration value from a to b with function.

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx = function(x) exp(-x/100)
romb(fx, 0, 24, 10)
```

Round	<i>Ordinary Rounding Function</i>
-------	-----------------------------------

Description

Rounding function in R is round and its behaviour is different from other common software. This is an ordinary rounding function.

Usage

```
Round(x, digits=0)
```

Arguments

x	real number to be rounded
digits	integer indicating the number of decimal places. 0 means $10^0=1$, 2 means $10^{-2}=0.01$, -1 means $10^1 = 10$ etc.

Details

Ordinary rounding function (round-up). For more details, see wikipedia.

Value

rounded value

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
Round(1:10 - 0.5)
Round(450, -2)
Round(1.235, 2)
```

run.p	<i>Probability for Run Test</i>
-------	---------------------------------

Description

p-value for run test

Usage

run.p(m, n, r)

Arguments

m	count of fewer species (minimum value = 0)
n	count of more frequent species (minimum value = 1)
r	count of run (minimum value = 1)

Details

It calculates p-value for run test, $P(\text{Run count} \leq r \mid m, n)$

Value

probability of run count to be less than or equal to r with m and n. $P(\text{Run count} \leq r \mid m, n)$

Author(s)

Kyun-Seop Bae <k@acr.kr>

Run.test	<i>Probability for Run Test</i>
----------	---------------------------------

Description

p-value for run test

Usage

Run.test(RES)

Arguments

RES	sequence vector to be tested, usually residual values)
-----	--

Details

It calculates p-value for run test. Zeros are omitted first.

Value

p-value, if it is larger than 0.5, it returns 1 - p.

Author(s)

Kyun-Seop Bae <k@acr.kr>

runifLC

Random number of uniform distribution [0, 1) by linear congruential method

Description

Random number generation distributed as Uniform[0, 1) by linear congruential method

Usage

```
runifLC(n = 1, Seed = 123457, a = 16807, m = 2147483647, k = 0, b = 10)
```

Arguments

n	count of random numbers to generate
Seed	Seed number. If abs(Seed) is less than 1, Sys.time() is the seed.
a	number to multiplier
m	number to mod. It should be a prime number.
k	number to add
b	number to discard initially

Details

$R[i] = X[i]/(m + 1)$. $X[i] = (a * X[i - 1] + c) \bmod m$. Then, period is $m - 1$.

Value

Uniform[0, 1) random numbers of n

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
runifLC(8)
runifLC(8, 1)
runifLC(8, 2)
runifLC(8, 0) # random seed
```

sft	<i>Slow Fourier Transformation</i>
-----	------------------------------------

Description

Fourier transformation using the definition, therefore slow.

Usage

sft(y)

Arguments

y a sequence of fixed time interval

Details

This is slow but easy to understand.

Value

Fourier transformation result

Author(s)

Kyun-Seop Bae <k@acr.kr>

simps13	<i>Integration using Simpson's 1/3 formula</i>
---------	--

Description

Integrates with given fx, a, b, and n

Usage

simps13(fx, a, b, n)

Arguments

fx	real valued scalar function
a	from
b	to
n	split count

Details

This gives approximate integration value using Simpson's 1/3 formula. Argument n should be even number.

Value

Integration value from a to b with function.

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx = function(x) exp(-x/100)
simps13(fx, 0, 24, 10)
```

simps38

Integration using Simpson's 3/8 formula

Description

Integrates with given fx, a, b, and n

Usage

```
simps38(fx, a, b, n)
```

Arguments

fx	real valued scalar function
a	from
b	to
n	split count

Details

This gives approximate integration value using Simpson's 3/8 formula. Argument n should a multiple of 3.

Value

Integration value from a to b with function.

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx = function(x) exp(-x/100)
simps38(fx, 0, 24, 10)
```

Sqrt*Square-rooting*

Description

Square-rooting of a real positive number.

Usage

Sqrt(x)

Arguments

x a real positive number to be square-rooted

Details

Ref. Plauger PJ The Standard C Library. 1992.

Value

square-rooted value

Author(s)

Kyun-Seop Bae <k@acr.kr>

SymDet*Determinant of a Symmetric Matrix*

Description

Determinant of a symmetric matrix

Usage

SymDet(SymMat)

Arguments

SymMat a symmetric matrix of real values

Details

Input should be a symmetric matrix of real values.

Value

Determinant of the input

Author(s)

Kyun-Seop Bae <k@acr.kr>

SymInv	<i>Inverse of a Symmetric Matrix</i>
--------	--------------------------------------

Description

Inverse of a symmetric matrix

Usage

SymInv(SymMat)

Arguments

SymMat	a symmetric matrix of real values
--------	-----------------------------------

Details

Input should be a symmetric matrix of real values.

Value

Inverse of the input

Author(s)

Kyun-Seop Bae <k@acr.kr>

SymSol	<i>Solution of a Symmetric Matrix</i>
--------	---------------------------------------

Description

Solution of a symmetric matrix linear equation

Usage

SymSol(H, g)

Arguments

H	a symmetric matrix of real values, such as hessian
g	a response vector, such as gradient

Details

Input should be a symmetric matrix of real values and a real vector

Value

Solution of the linear system with a real symmetric coefficient matrix

Author(s)

Kyun-Seop Bae <k@acr.kr>

tableFactorial	<i>Factorial using Table</i>
----------------	------------------------------

Description

This function returns factorial value from the stored table.

Usage

```
tableFactorial(n)
```

Arguments

n	positive integer to be calculated
---	-----------------------------------

Details

This function returns factorial value from the table known to be precise.

Value

factorial value

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
tableFactorial(0)
tableFactorial(170)
tableFactorial(171)
```

trapez0	<i>Integration using trapezoidal rule</i>
---------	---

Description

Integrates with given two sequences: x_i and y_i

Usage

```
trapez0(x, y)
```

Arguments

x	sequence of x
y	sequence of y

Details

This simplest implementation of trapezoidal rule. The length of two sequences should be same

Value

Integration value with given sequences.

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx = function(x) exp(-x/100)
x = c(0, 0.5, 1, 2, 3, 4, 6, 8, 12, 24)
y = fx(x)
trapez0(x,y)
```

trapez1

Integration using trapezoidal rule

Description

Integrates with given fx, a, b, and n

Usage

```
trapez1(fx, a, b, n)
```

Arguments

fx	real valued scalar function
a	from
b	to
n	split count

Details

This gives approximate integration value using trapezoidal rule.

Value

Integration value from a to b with function.

Author(s)

Kyun-Seop Bae <k@acr.kr>

Examples

```
fx = function(x) exp(-x/100)
trapez1(fx, 0, 24, 10)
```

VMmin

Simplified version of optimization function

Description

To show the algorithm how quasi-Newton type optimization algorithm works.

Usage

```
VMmin(x0, func, MaxIter=9999, Tol=1e-4)
```

Arguments

x0	initial value
func	function to be minimized
MaxIter	iteration count limit
Tol	tolerance

Details

If the changes of between steps are less than the tolerances, it stops.

Value

par	final x vector values
value	function value at the final x vector
FnCount	function evaluation count
GrCount	gradient evaluation count
convergence	convergence is met or not
grad	gradient at the final x vector

Author(s)

Kyun-Seop Bae <k@acr.kr>

Index

* internal

- betacf, 3
- betaiapprox, 5
- gammapprox, 29
- gcf, 30
- gser, 34

- betacf, 3
- betai, 4
- betaiapprox, 5
- Bin2Dec, 6

- ChkFx, 7
- Chol, 8
- Choose, 9
- conv, 9
- conv0, 10

- Dbeta, 11
- Dbinom, 11
- dc, 12
- Dchisq, 13
- Dec2Bin, 13
- deconv, 14
- deconv0, 15
- DENORM, 15
- Deriv0, 16
- Deriv1, 16
- Deriv2, 17
- Df, 18
- Dgamma, 18
- Dlnorm, 19
- Dnorm, 20
- Dpois, 20
- Dt, 21

- ellipRange, 22
- ellipse, 22
- erf, 23
- erfc, 24
- erfccheb, 25
- evJacobi, 26
- EXP, 26

- GAMMA, 27

- gammln, 28
- gamp, 28
- gammapprox, 29
- gammq, 30
- gcf, 30
- GenD, 31
- GetP, 32
- GQuad8, 32
- Grad, 33
- gser, 34

- Hessian, 34

- invbetai, 35
- inverfc, 36
- invgamp, 36

- LDLT, 37
- LGAMMA, 38
- LOG, 38

- MachEps, 39
- MachEps2, 40
- MachEps3, 40
- MachEps4, 41
- mlr, 42

- NashSVD, 43
- NORM, 43

- Optim0, 44
- OuterProd, 45

- Pbeta, 45
- Pbinom, 46
- Pchisq, 46
- Pf, 47
- Pgamma, 48
- Plnorm, 48
- Pnorm, 49
- PolyNom3, 49
- Ppois, 50
- Pt, 51

- Qbeta, 51

Qbinom, [52](#)
Qchisq, [52](#)
Qf, [53](#)
Qgamma, [54](#)
Qlnorm, [54](#)
Qnorm, [55](#)
Qpois, [55](#)
Qt, [56](#)

Rbeta, [57](#)
Rexp, [57](#)
Rgamma, [58](#)
Rgamma0, [59](#)
RK4, [59](#)
Rmvn, [60](#)
Rnorm, [61](#)
romb, [61](#)
Round, [62](#)
run.p, [63](#)
Run.test, [63](#)
runifLC, [64](#)

sft, [65](#)
simps13, [65](#)
simps38, [66](#)
SQRT, [67](#)
SymDet, [67](#)
SymInv, [68](#)
SymSol, [68](#)

tableFactorial, [69](#)
trapez0, [69](#)
trapez1, [70](#)

VMmin, [71](#)